FlagPrism: Compiler-Assisted Fine-Grained Profiling Across AI Accelerators (Operational Systems)

Anonymous Authors

Abstract

[TODO: Write the abstract after confirming the study findings, system mechanisms, verified hardware coverage, and operational experience. State measured results only after experiments; the working title is provisional.**]**

1 Introduction

Unified kernel programming frameworks allow operator developers to reuse program logic across artificial intelligence (AI) accelerators. For example, FlagTree extends the Triton ecosystem to multiple accelerator backends [1]. Obtaining high performance still requires understanding how a target executes that logic: where computation, data movement, and synchronization consume time; whether these activities overlap; and how tiling, layout, and pipeline choices change execution. These profiling questions recur across devices, even when their answers require different execution objects and measurement mechanisms. A useful profiler must connect the resulting observations to the operator implementation being optimized.

Architecture diversity appears at more than one level. Graphics processing units (GPUs) with similar single-instruction, multiple-thread (SIMT) execution models can differ in thread organization, local-memory structure, resource limits, and asynchronous facilities [6, 7]. Accelerators with dedicated computation and data-movement pipelines introduce further differences in how operations are scheduled and synchronized [4]. Such differences affect profiling as well as optimization. A source region can be decomposed, fused, reordered, or pipelined during compilation; timing the issue of an asynchronous operation need not measure its execution or completion. A common interface or output format therefore does not, by itself, determine which execution events should be observed, where instrumentation belongs, or how records should be related back to a program region.

Existing work supplies important foundations. KPerfIR integrates fine-grained profiling into compiler intermediate representation (IR) and analyzes intra-kernel execution and overlap [2]; Neutrino supports programmable instruction-level probes [3]. Proton combines vendor measurement interfaces and instrumentation with multi-level program context [9], while PASTA provides a modular analysis framework for accelerator workloads [5]. These capabilities motivate a concrete question for a broader collection of execution backends: which program mappings, instrumentation semantics, and measurement assumptions can be reused, and which must change when the target architecture changes? **[TODO:** Verify the exact supported versions and mechanisms of each baseline, and replace this scoped question with evidence-supported limitations from the study. Avoid claiming unsupported platforms are inherently impossible to support.**]**

This paper investigates the portability of *fine-grained profiling intent*: a developer's request to observe a logical operator region or an execution behavior relevant to optimization. We separate three difficulties. First, compilation changes the relationship between logical regions and execution instances. Second, instrumentation must respect the target's execution semantics, including the distinction between submission, execution, and waiting. Third, timestamps, recording mechanisms, and vendor observations must be reconstructed into program-associated results without hiding architecture-specific detail. These difficulties occur both within a broadly shared execution model and across substantially different execution models.

We place an empirical study before the system design. Using NVIDIA V100S, Enflame GCU300, and Ascend 910B4-1, it follows common operator logic into compiled execution objects, tests a shared nested-region timing request, and checks a controlled tiling intervention with native profiling detail. Different source-to-execution mappings and counter protocols motivate explicit backend contracts; useful vendor-specific detail motivates preserving native observations. Differences in device speed alone do not establish a profiling limitation. The study concerns these backend/software-stack combinations and does not establish a same-SIMT-model comparison across vendors.

We investigate FlagPrism, a profiler whose system boundary includes the profiling-related compilation mechanisms in FlagTree, together with runtime collection, event association, and analysis. Its intended service is to lower common profiling requests into target-appropriate observations and reconnect those observations to the operator program. It provides profiling services to operator developers and optimization tools. The compiler–profiler boundary and its concrete mechanisms will be defined from the study rather than fixed in advance. **[TODO:** Finalize whether profiling intent requires a new dialect, extensions to existing IR, metadata and passes, or a combination. Distinguish inherited compiler/profiler functionality, implemented extensions, and proposed work.**]**

The planned contributions are:

- An empirical characterization of fine-grained profiling portability across three accelerator backend stacks. **[TODO:** Replace the planned characterization with verified findings and scope.**]**
- Compiler–profiler mechanisms for program mapping, target-specific instrumentation, and program-associated reconstruction. **[TODO:** Name and substantiate only the mechanisms actually implemented.**]**
- A system evaluation and operational account of profiling-guided operator optimization, including measurement fidelity, perturbation, unsupported observations, and unsuccessful diagnoses. **[TODO:** Supply actual case studies and usage records; no deployment or performance claim is established yet.**]**

2 Background and Profiling Scope

2.1 Operators, Backends, and Execution Models

[TODO: Define the operator programming model, supported input/configuration space, backend compiler versions, and concrete chip models. Separate vendor, architecture generation, execution model, and software stack. Describe NVIDIA, Iluvatar, and Ascend at verified granularity for the study; introduce additional vendors only as needed for system evaluation.**]**

2.2 From Program Regions to Device Execution

A logical region is a developer-visible part of an operator. An execution instance is the concrete work associated with a particular invocation and configuration. Their correspondence may be one-to-many or many-to-one. **[TODO:** Choose the exact identity and provenance model after inspecting backend IR; define treatment of fusion, duplication, loops, and overlapping regions.**]**

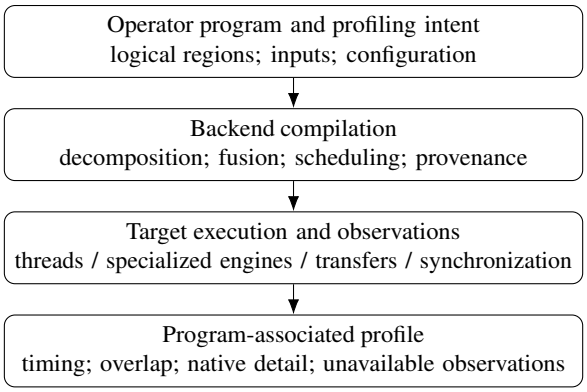


Figure 1: Conceptual profiling scope. The branches illustrate possible mappings, not measured schedules or a claim about a particular chip. The study will replace them with compiler-derived and runtime-validated examples.

2.3 Measurement Semantics

[TODO: Define host submission, device execution, asynchronous completion, waiting, overlap, timestamp domains, and estimated versus measured quantities. State which quantities cannot be observed on each tested environment.**]**

3 Study on Cross-Architecture Profiling

Logic. Program/execution differences → consequences for the same profiling request → controlled checks → compiler–profiler requirements.
Scope. NVIDIA V100S, Enflame GCU300, Ascend 910B4-1; six fixed-source probes; three shapes and two configurations per probe. Byte-identical inputs → CPU-reference checks → saved compiler artifacts → separate baseline/native captures. All 216 cases pass; 1,620 native intervals and 1,296 artifact hashes are audited. These are backend-stack observations, with neither fixed clocks nor exclusive hardware. The sample ranges are descriptive, not confidence intervals.

3.1 Program and Execution Differences

Breadth. GEMM / fused GEMM+bias+GELU / softmax / layernorm / reduce-sum / transpose → different duration distributions and configuration responses. Duration differences motivate inspection; they do not establish a profiling defect or a hardware-only cause.
Intervention check. Requested GEMM stages1/2 → identical GCU and V100 final binaries at all three shapes; changed Ascend binaries and emitted buffering/events. A shared parameter name does not establish an equivalent compiled intervention.
Case 1: execution objects. One fused source region → one V100 PTX entry using FP32 FMA in this compiler lowering;

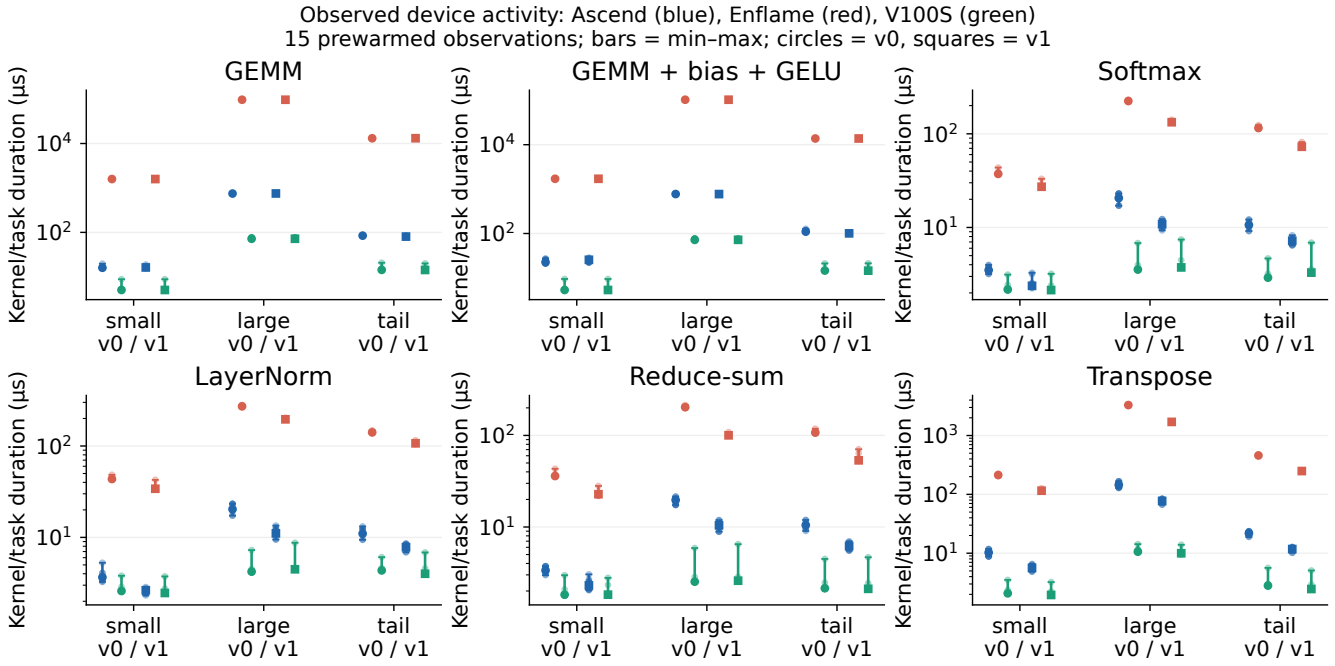


Figure 2: Six untuned probes, three shapes and two configurations per backend. Each configuration has 15 prewarmed vendor-reported kernel/task intervals; small dots show observations, larger circle/square marks show medians, and bars show minimum–maximum. GEMM shapes: 128^3, 512^3, $257 \times 193 \times 320$; row shapes: 32×256, 256×1024, 127×769; transpose shapes: 256^2, 1024^2, 513×257. v0/v1 request stages1/2 (GEMM), row grouping1/2, or transpose tile M16/32. Different backend software stacks are part of this comparison.

one GCU kernel with local matmul and epilogue; Ascend AIC/AIV functions with workspace and explicit set/wait dependency → one aggregate native kernel/task interval per launch. A source region, a transformed function, and an activity record are different objects. Static dependencies do not measure overlap or internal wait time.

Pipeline structure. Data movement → compute → epilogue/output, with different memory spaces and synchronization objects (Figure 4). In the Ascend lowering, AIC produces a workspace tile and signals AIV; AIV waits, loads, and runs the epilogue. Thus a request for "epilogue elapsed time" must specify whether its scope includes the cross-core wait, workspace load, or vector computation. The figure establishes these distinct boundaries; their dynamic durations and overlap remain unmeasured.

3.2 Consequences for the Same Profiling Request

Separate protocol probe. Case 1 establishes a mapping/scope requirement; the following probe isolates timing protocol behavior rather than measuring that fused kernel.

Case 2: measurement protocol. Request: elapsed interval of an outer logical region and its inner subregion. Same vector recurrence and correct final output → generic nested begin/end subtraction → materially different interval semantics. V100 uses `globaltimer`; Ascend uses `SYS_CNT`; the GCU backend provides `begin_clock/end_clock`. Similar API spelling does not establish that all three behave as independently readable timestamps.

Controlled check. Outer: 256 dependent FMA iterations; inner: 128. Move q iterations before the inner region and $128 - q$ after it → hold total work and output fixed → capture outer and inner separately with only one active counter region → compare each with the nested capture (Figure 5). The GCU outer nested result is approximately half its single-region reference; its inner result remains close. Ascend and V100 results remain close to their single-region references across q.

Endpoint check. Fixed input-readiness barrier; vary only an end barrier; include zero-work controls → observe changes in reported brackets (Figure 6). Barrier overhead and execution completion are not separated by this experiment. Counter units remain native: GCU units are not converted to nanoseconds without a verified conversion.

Useful detail. GCU GEMM and fused GEMM, fixed 128^3 input and BK=32; change output tile $16^2 \rightarrow 64^2$ → shorter native intervals in counter-free captures; TCU captures expose different instruction-family efficiencies → preserve device, invocation, and per-SIP detail rather than reducing observa-

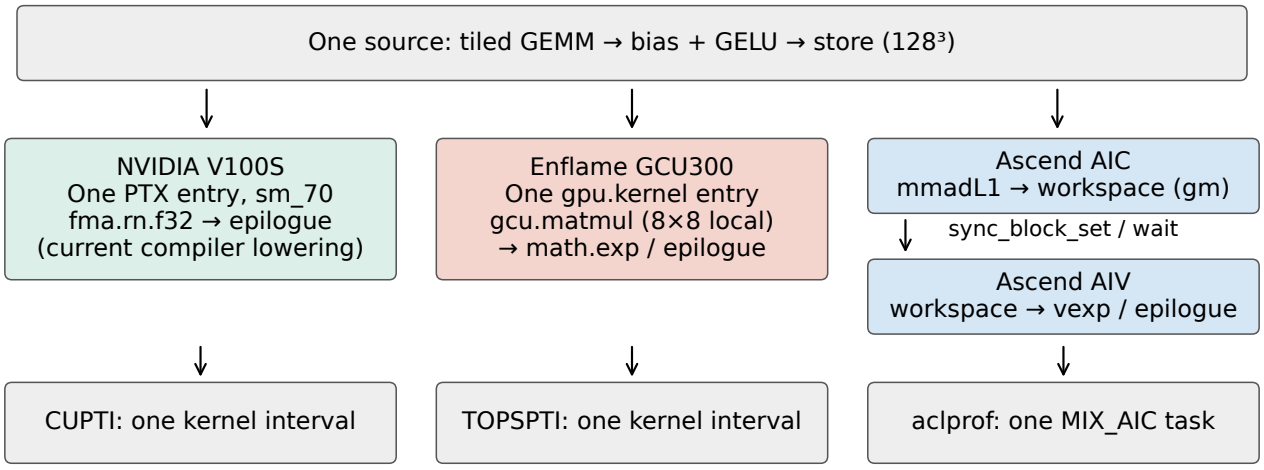


Figure 3: Actual emitted structure for fused GEMM+bias+GELU (128^3), alongside native observation scope. This is a compiler mapping diagram, not a measured internal timeline. The V100 lowering shown uses FMA; it is not a claim about V100's best implementation. Ascend diagnostic/baseline binary equivalence is checked in the saved evidence.

tions to a lowest-common-denominator schema. TCU already provides these details; our exporter preserves them. Efficiency families retain SDK definitions and may overlap. MSF is not identified as a matrix engine. Counter collection and counter-free timing are separate runs.

3.3 Requirements for Cross-Architecture Profiling

- Source-to-execution decomposition → retain logical regions, transformed functions, and execution units separately; expose mapping provenance.
- Different clock/recording protocols → backend contracts for region scope, nesting, endpoints, units, and unsupported requests.
- Useful native detail → preserve device/instance identity and backend-specific metrics; report which associations are established.

[TODO: Implement and evaluate the corresponding compiler–profiler mechanisms. The study does not yet measure the fused kernel's internal AIC/AIV busy/wait intervals, establish an attribution mismatch rate, or validate an adapted nested instrumentation mechanism. Additional vendors and operational use belong in evaluation; these three devices do not establish a same-SIMT multi-vendor result.**]**

4 Compiler-Assisted Profiling Design

[TODO: Tie each mechanism to a concrete consequence established in Sections 3.2 and 3.3. Freeze the system boundary before claiming design novelty.**]**

4.1 Profiling Intent and Backend Contracts

[TODO: Define requests, observations, supported capabilities, and the native extension mechanism. Decide between a new profiling IR and extensions to existing IR/metadata. Specify semantics rather than only an API.**]**

4.2 Program Mapping Through Compilation

[TODO: Explain logical-region identity, transformation provenance, one-to-many and many-to-one relationships, and attribution after fusion and scheduling.**]**

4.3 Execution-Aware Instrumentation

[TODO: Describe instrumentation placement and lowering for each execution model. Distinguish issue, completion, and waits; state unsupported quantities. Explain how instrumentation interacts with optimization and synchronization.**]**

4.4 Collection and Reconstruction

[TODO: Define timestamp calibration, buffering, event association, lost-record handling, concurrency, and the separation

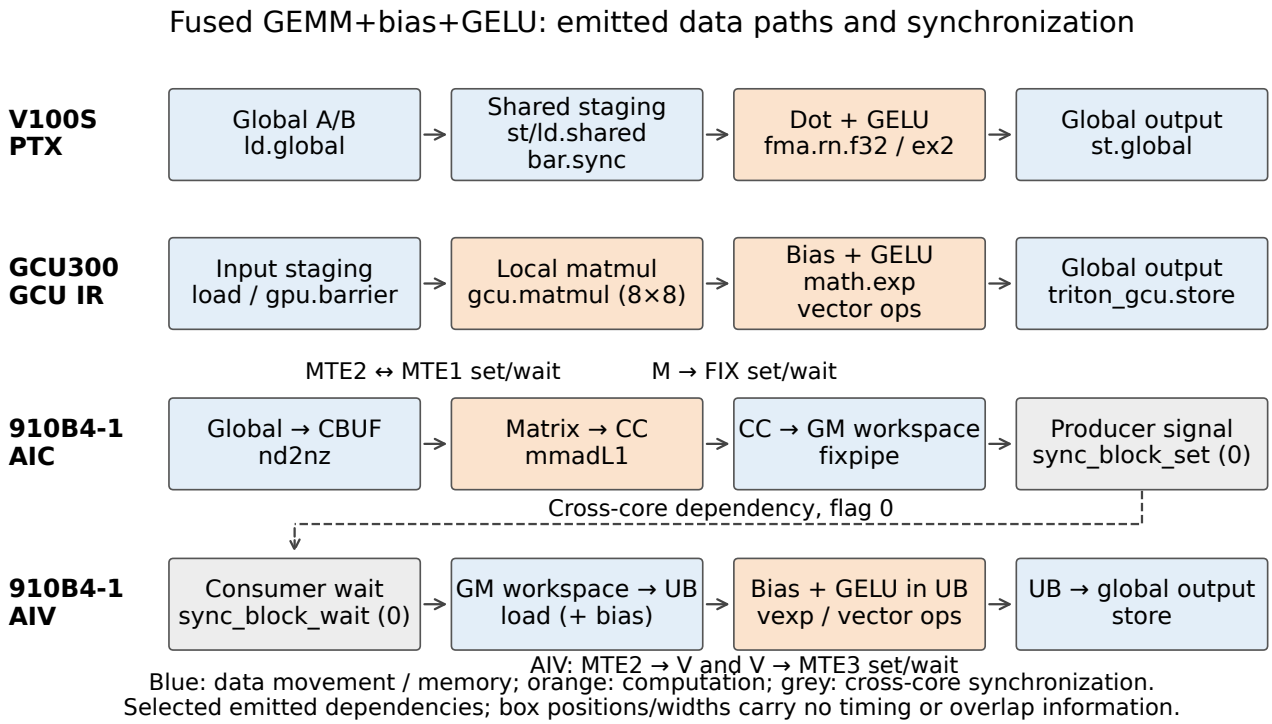


Figure 4: Selected pipeline stages and dependencies extracted from the actual fused-kernel PTX/GCU/HIVM outputs. Arrows denote data/program dependencies; the dashed arrow denotes the Ascend cross-core set/wait relation, flag 0. Layout and box widths are not a time axis. Explicit AIC/AIV memory paths and synchronization are compiler evidence; this is not an internal execution trace or a complete microarchitectural pipeline model.

of measurements from static or derived relationships. Preserve backend-specific observations.]

5 Implementation

[TODO: Record pinned FlagTree/FlagPrism commits and backend toolchains. Document compiler passes, instrumentation runtime, vendor collectors/importers, and profile outputs. Quantify shared versus backend-specific implementation without treating line count as proof of abstraction quality.]

5.1 Backend Integration and Capability Status

[TODO: Use a per-chip, per-SDK acceptance matrix. Distinguish an API skeleton, a real capture, successful program association, and fidelity validation. A readable report or passing smoke test does not establish full support.]

5.2 Limitations and Failure Handling

[TODO: Specify unavailable clocks/counters, permissions, replay constraints, unsupported IR transformations, ambiguous matches, and explicit degradation.]

6 System Evaluation

The study in Section 3 characterizes the problem. This section will evaluate the proposed mechanisms on separate or held-out tasks and additional backends beyond NVIDIA, Iluvatar, and Ascend. **[TODO:** Freeze study/evaluation separation and avoid selecting evaluation cases only after seeing favorable study results.]

6.1 Measurement and Attribution Fidelity

[TODO: Validate event coverage, region association, timestamp accuracy, and reconstruction against independent reference observations and controlled microbenchmarks. State when a reference is not independent.]

6.2 Overhead and Execution Perturbation

[TODO: Report per-mode and per-device latency, memory/register use, trace volume, processing cost, and changes to compiled schedules. Check bottleneck/ranking changes caused by instrumentation.]

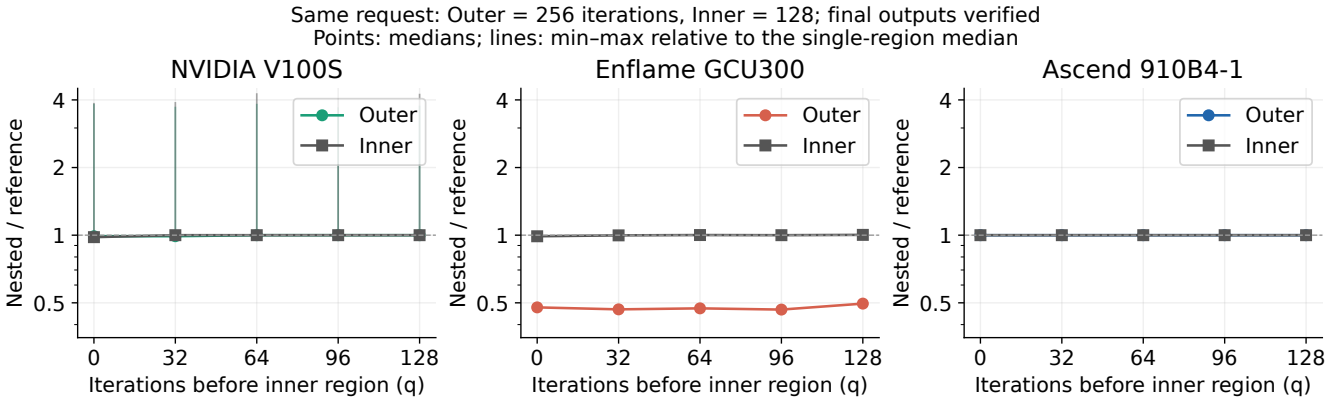


Figure 5: A common nested subtraction baseline versus separately captured single-region brackets. All probes pass final-output checks; each median uses 15 observations. Vertical spans show nested-capture minimum–maximum divided by the single-region reference median; CUDA outliers are retained. The ratio axis is logarithmic. References use the same vendor clock primitive in separate captures, not an independent physical clock. Instrumentation and barriers perturb execution; results concern bracket semantics, not unperturbed busy time. This baseline is not a reproduction of Proton, KPerfIR, or TCU.

6.3 Profiling-Guided Operator Optimization

[TODO: For each case record the initial hypothesis, additional observation, intervention, code change, correctness check, and profiler-disabled performance. Compare against native tools and reasonable target-local implementations.]

6.4 Mechanism Ablations and Backend Extension

[TODO: Remove transformation provenance, execution-aware placement, or native observation detail separately. Show what information or correctness is lost. Measure integration changes for a backend added after common analyses are frozen.]

6.5 Additional Backends

[TODO: Validate actual supported modes on Moore Threads, Enflame, MetaX, and other available devices. Report per-backend fidelity, overhead, native detail, and limitations rather than extending the three-backend Study.]

7 Operational Experience and Lessons

[TODO: Provide actual adoption context, duration, operators, developer workflows, profiling sessions, resolved/unresolved issues, maintenance burden, and examples of failed hypotheses. The project has not yet supplied bulk usage evidence.]

7.1 Lessons Across Similar and Different Execution Models

[TODO: Derive lessons from cases rather than restating architecture descriptions. Discuss which mechanisms were reusable and where the abstraction broke.]

7.2 Scope and Threats to Validity

[TODO: Discuss device coverage, compiler/SDK confounding, finite search spaces, measurement perturbation, missing references, and application generalization.]

8 Related Work

Compiler and instruction-level profiling. KPerfIR [2], Neutrino [3], and Proton [9] provide compiler, probe, and multi-level profiling foundations. **[TODO:** Compare concrete supported mechanisms and versions; do not attribute common-counter-only limitations to these systems.]

Context and accelerator analysis. DeepContext [8] connects program contexts across the deep learning stack, and PASTA [5] supplies modular accelerator analysis. **[TODO:** Complete comparisons with HPCToolkit, XProf, RL-Scope, Hotline, Snoopie, DrGPUM, and TenProf using the existing-works survey and primary papers. Separate broader workload diagnosis from our operator profiling scope.]

9 Conclusion

[TODO: Summarize established study findings, implemented mechanisms, measured benefits, and limitations after the em-

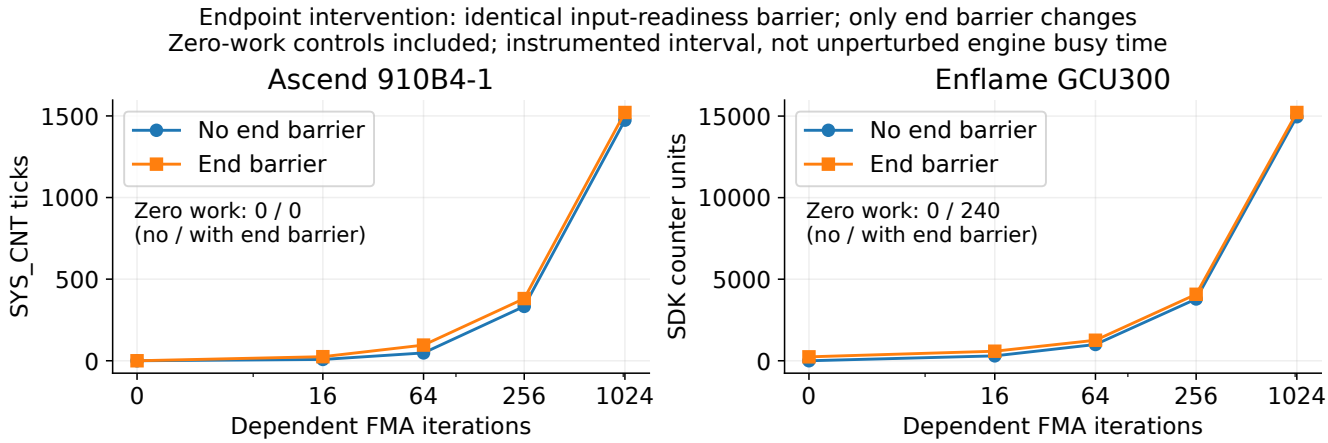


Figure 6: Single-region endpoint intervention, including zero-work controls. Points are medians of 15 samples per setting. The work axis uses a symmetric logarithmic scale (linear through 16); axes retain backend units. These are instrumented interval measurements.

pirical work is complete.]

References

[1] FlagOS. FlagTree: A unified compiler for multiple ai chip backends. Project repository. Accessed October 12, 2026. https://github.com/flagos-ai/FlagTree.

[2] Yue Guan, Yuanwei Fang, Keren Zhou, Corbin Robeck, Manman Ren, Zhongkai Yu, Yufei Ding, and Adnan Aziz. KPerfIR: Towards a open and compiler-centric ecosystem for GPU kernel performance tooling on modern AI workloads. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 205–220. USENIX Association, 2025. https://www.usenix.org/conference/osdi25/presentation/guan.

[3] Songlin Huang and Chenshu Wu. Neutrino: Fine-grained GPU kernel profiling via programmable probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 331–355. USENIX Association, 2025. https://www.usenix.org/conference/osdi25/presentation/huang-songlin.

[4] Huawei. AI Core architecture. Ascend CANN documentation. Accessed October 12, 2026. https://www.hiascend.com/doc_center/source/en/canncommercial/800/opdevg/tbeaicpudevg/atlasopdev_10_0008.html.

[5] Mao Lin, Hyeran Jeon, and Keren Zhou. PASTA: A modular program analysis tool framework for accelerators. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. https://2026.cgo.org/details/cgo-2026-papers/28/PASTA-A-Modular-Program-Analysis-Tool-Framework-for-Accelerators.

[6] Moore Threads. MUSA hardware architecture. MUSA SDK 4.3.x programming guide. Historical version; architecture statements apply to its documented devices. Accessed October 12, 2026. https://docs.mthreads.com/musa-sdk/version-4.3.x/programming_guide/Chapter02/.

[7] NVIDIA. Hopper tuning guide. CUDA documentation. Accessed October 12, 2026. https://docs.nvidia.com/cuda/archive/13.2.0/hopper-tuning-guide/index.html.

[8] Qidong Zhao, Hao Wu, Yueming Hao, Zilingfeng Ye, Jiajia Li, Xu Liu, and Keren Zhou. Deepcontext: A context-aware, cross-platform, and cross-framework tool for performance profiling and analysis of deep learning workloads. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 48–63, 2025. https://arxiv.org/abs/2411.02797.

[9] Keren Zhou, Tianle Zhong, Hao Wu, Jihyeong Lee, Yue Guan, Yufei Ding, Corbin Robeck, Yuanwei Fang, Jeff Niu, and Philippe Tillet. Proton: Towards multi-level, adaptive profiling for triton. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. https://2026.cgo.org/details/cgo-2026-papers/23/Proton-Towards-Multi-level-Adaptive-Profiling-for-Triton.

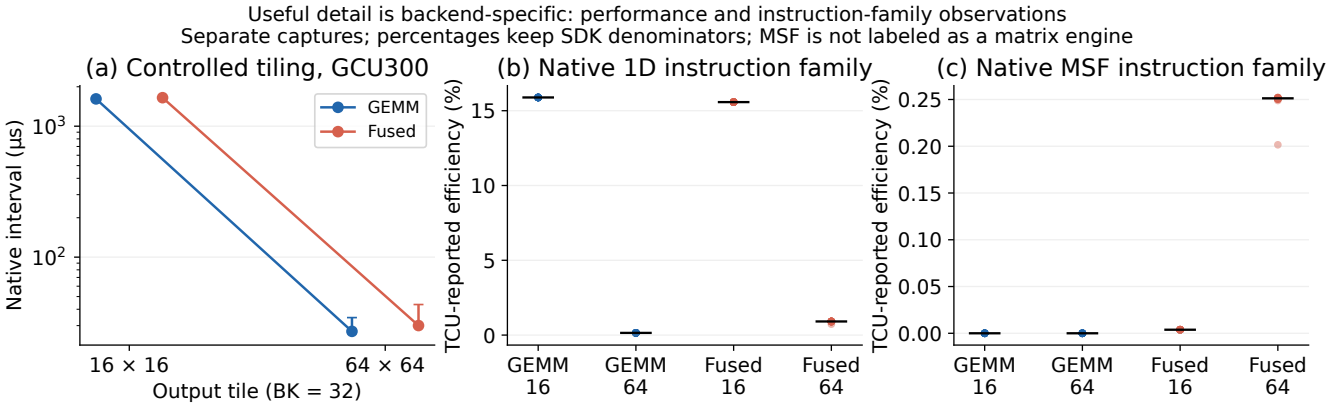


Figure 7: Controlled tiling on GCU300. Left: 15 native intervals per configuration without TCU counters (points: medians; bars: minimum–maximum). Fused denotes GEMM+bias+GELU. Right: all 21 TCU invocations, including warmup, with backend-defined efficiencies. Invocation IDs are capture-local; no join to TOPSPTI activity is inferred from matching kernel names. These observations check a concrete intervention, not a complete bottleneck attribution. Black marks show medians. Per-SIP details are retained in the artifact; these panels show invocation aggregates. 1D counts M/V-slot instruction cycles relative to total cycles; MSF counts M-slot MSF instruction cycles relative to total cycles (artifact definitions retain the SDK semantics).

[TODO: Draw the final compiler–runtime–analysis architecture, marking inherited components and the profiling-specific extensions introduced by this work.]

Figure 8: System architecture placeholder; mechanisms and boundaries remain to be determined.